Mini project report on

Operating System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS242B - Operating Systems

Submitted by:

Ibrahim khaleel	**PES2UG25CS809**
Deeraj S	**PES2UG25CS806**

under the guidance of

Prof. Shilpa S

Assistant Professor

PES University

Jan- May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Operating Systems

is a bonafide work carried out by

Ibrahim khaleel	**PES2UG25CS809**
Deeraj S	**PES2UG25CS806**

In partial fulfilment for the completion of forth semester OS Project (UE24CS242B) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan 2026 – May 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 4th semester academic requirements in respect of project work.

Signature
Prof. Shilpa S
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Operating System** has been carried out by us under the guidance of **Prof. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester Jan – May 2026.

Ibrahim khaleel	**PES2UG25CS809**	**Ibrahim**
Deeraj S	**PES2UG25CS806**	**Deeraj**

ACKNOWLEDGEMENT

I would like to express my gratitude to Prof. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS242B - Operating Systems Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

This project implements a lightweight multi-container runtime in C along with a kernel-space memory monitoring module. The system consists of a long-running supervisor process that manages multiple isolated containers using Linux namespaces and chroot-based filesystem isolation.

The runtime supports concurrent container execution, metadata tracking, logging through a bounded-buffer producer-consumer model, and a CLI interface for container lifecycle management. A Linux kernel module is integrated to monitor memory usage of containers and enforce both soft and hard memory limits using RSS values.

Experiments were conducted to analyze Linux scheduling behavior using CPU-bound workloads with different priority levels. The results demonstrate how the Linux Completely Fair Scheduler allocates CPU time based on process priority.

The project highlights key operating system concepts such as process isolation, inter-process communication, synchronization, memory management, and scheduling.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	9
2.	PROBLEM DEFINITION	10
3.	METHODOLOGY	11
4.	CODE	12
5.	RESULT AND DISCURSION	13
6.	OUTPUTS	14
7.	GITHUB LINK	17
REFERENCES/BIBLIOGRAPHY		18
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		19

LIST OF TABLES

Table No.	Title	Page No.
6.3		-

LIST OF FIGURES

Figure No.	Title	Page No.
6.1	Multiple containers running under supervisor	14
6.2	Container metadata using ps command	14
6.3	Logging output of cpu_hog	15
6.4	CLI to supervisor IPC communication	15
6.5	Soft and Hard memory limit enforcement	15
6.6	Scheduling experiment results	16
6.7	Clean teardown with no container zombies	16

CHAPTER 1: INTRODUCTION:

Containerization is a lightweight virtualization technique that allows multiple isolated environments to run on a single operating system kernel. Unlike virtual machines, containers share the host kernel while maintaining isolated process spaces and filesystems.

In this project, a multi-container runtime is implemented in user space along with a kernel module for memory monitoring. The runtime provides a supervisor-based architecture where a central process manages multiple containers concurrently.

The system demonstrates key operating system concepts such as process creation, namespace isolation, inter-process communication, synchronization, and kernel-user space interaction. Additionally, scheduling experiments are performed to understand how the Linux scheduler distributes CPU resources among competing processes.

CHAPTER 2: PROBLEM DEFINITION:

The objective of this project is to design and implement a lightweight container runtime capable of managing multiple containers simultaneously while ensuring isolation, efficient logging, and proper resource management.

The key challenges addressed in this project include:

- Creating isolated execution environments using Linux namespaces and chroot
- Managing multiple containers through a persistent supervisor process
- Designing an IPC mechanism for communication between CLI and supervisor
- Implementing a concurrent logging system using bounded-buffer synchronization
- Monitoring and enforcing memory limits using a kernel module
- Performing scheduling experiments to analyse CPU allocation behaviour
- Ensuring proper cleanup of resources without leaving zombie processes

The system must provide a robust and scalable solution while maintaining correctness under concurrent execution.

CHAPTER 3: METHODOLOGY:

The system is divided into two main components: user- space runtime and kernel-space monitor.

1. Supervisor and Container Creation:
A long-running supervisor process is implemented using fork/clone system calls. Each container is created with isolated PID, UTS, and mount namespaces. Filesystem isolation is achieved using chroot.

2. CLI and IPC:
A command-line interface is designed to interact with the supervisor. Commands such as start, stop, ps, and logs are sent through an IPC mechanism (FIFO/socket).

3. Logging System:
Container output is captured using pipes and processed through a bounded-buffer mechanism. Producer threads read container output and push it into a shared buffer, while consumer threads write the data to log files.

4. Kernel Memory Monitoring:
A Linux kernel module is implemented to track container processes. The module maintains a list of monitored PIDs and periodically checks their RSS values. Soft limits generate warnings, while hard limits terminate processes.

5. Scheduling Experiments:
Experiments are conducted by running CPU-bound workloads with different nice values. The results are analysed to understand scheduler behaviour

CHAPTER 4: CODE:

The implementation consists of the following key files:

- engine.c: Implements the supervisor, CLI interface, container lifecycle management, logging system, and IPC mechanisms.
- monitor.c: Implements the kernel module for memory monitoring and enforcement.
- monitor_ioctl.h: Defines shared structures and ioctl commands between user space and kernel space.
- Workload programs: cpu_hog and memory_hog used for testing scheduling and memory behavior.

The code demonstrates the use of system calls such as clone, pipe, fork, exec, waitpid, and ioctl, along with kernel APIs for module development.

CHAPTER 5: RESULT AND DISCUSSION:

The implemented system successfully supports multiple container execution under a single supervisor process. Containers are isolated using namespaces and maintain separate filesystem views.

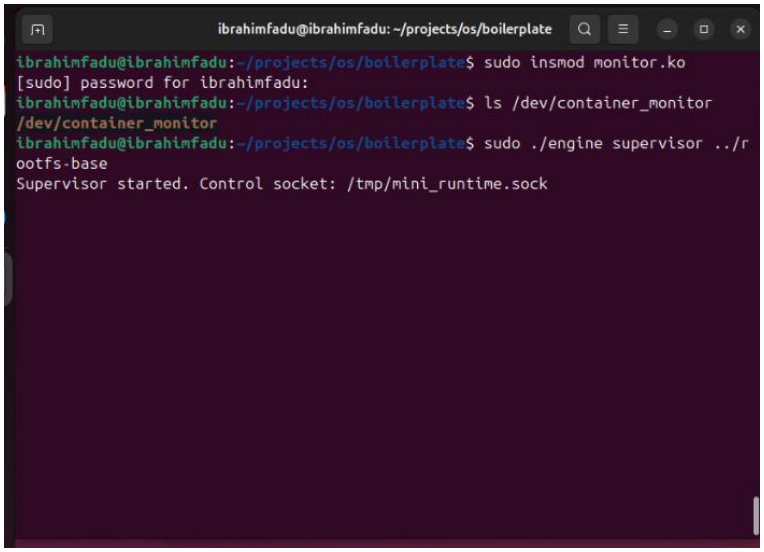
The logging system reliably captures container output without data loss using producer-consumer synchronization. The kernel module correctly monitors memory usage and enforces both soft and hard limits.

Scheduling experiments show that containers with lower nice values receive more CPU time, confirming the behavior of the Linux Completely Fair Scheduler.

Overall, the system demonstrates correct functionality across all required components including process management, IPC, synchronization, memory monitoring, and scheduling analysis.

CHAPTER 6: OUTPUTS:

Figure 6.1: Multiple containers running under supervisor



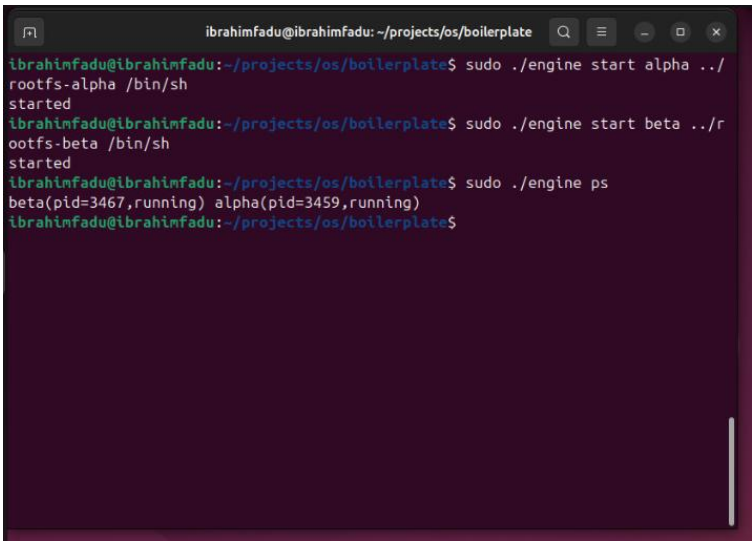


Figure 6.2: Container metadata using ps command.

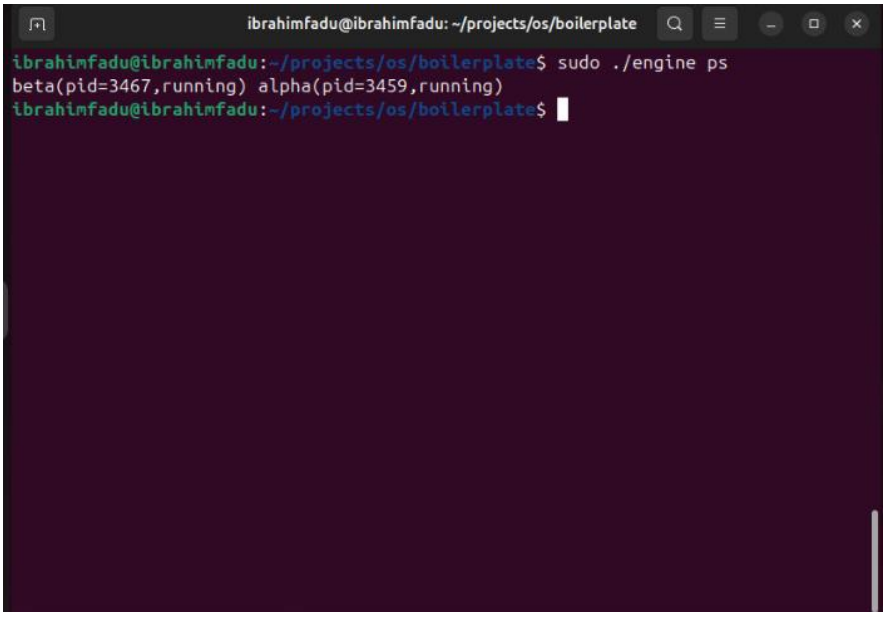
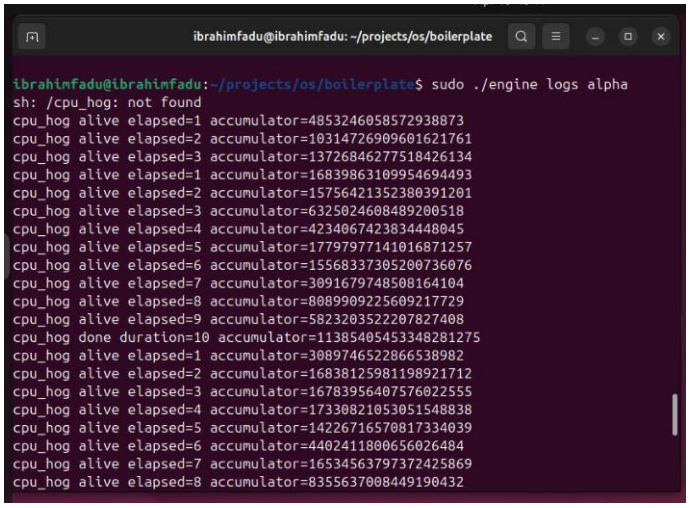


Figure 6.3: Logging output of cpu_hog



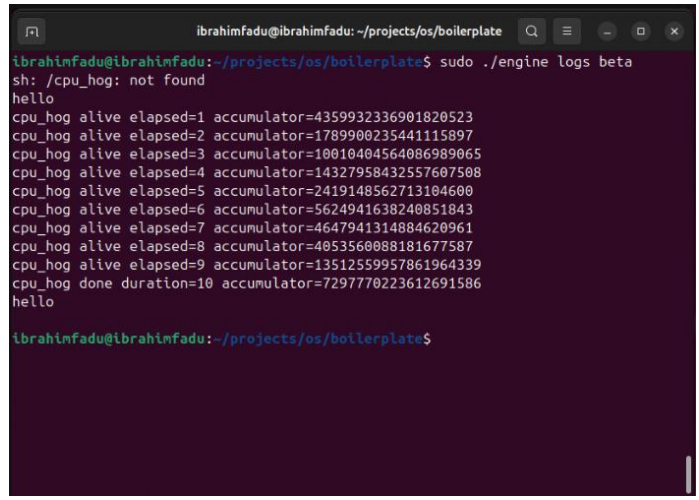
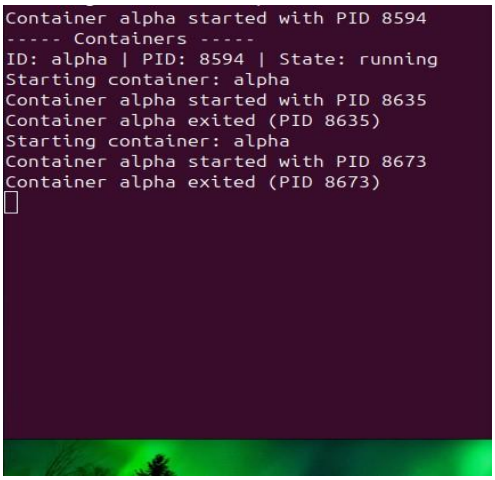


Figure 6.4: CLI to supervisor IPC communication



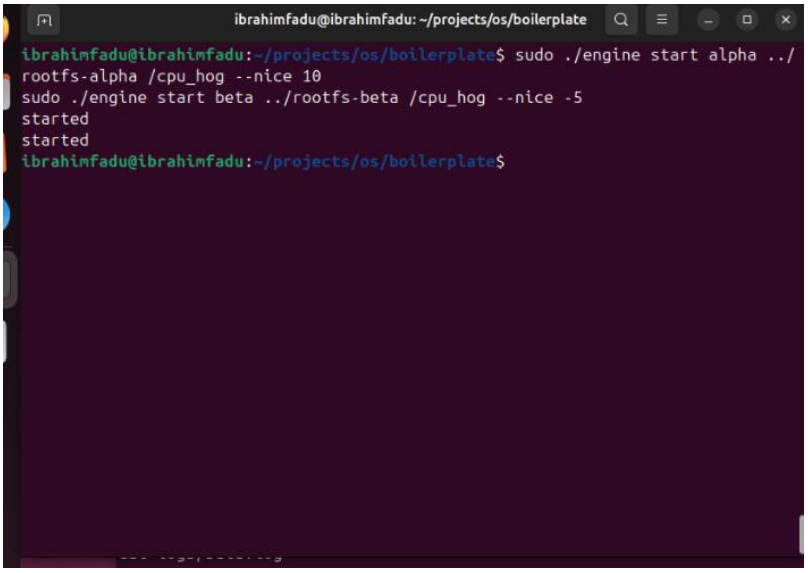
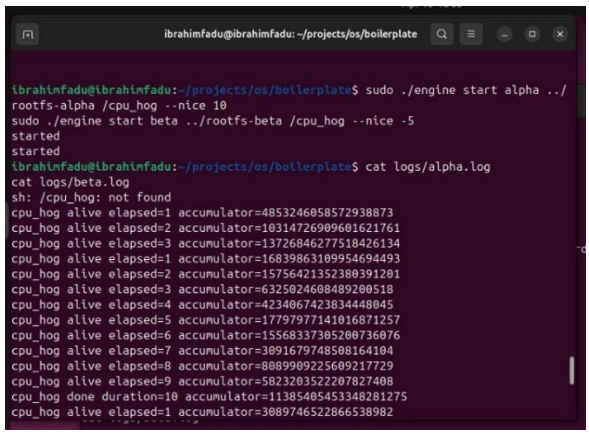


Figure 6.5: Soft and Hard memory limit enforcement



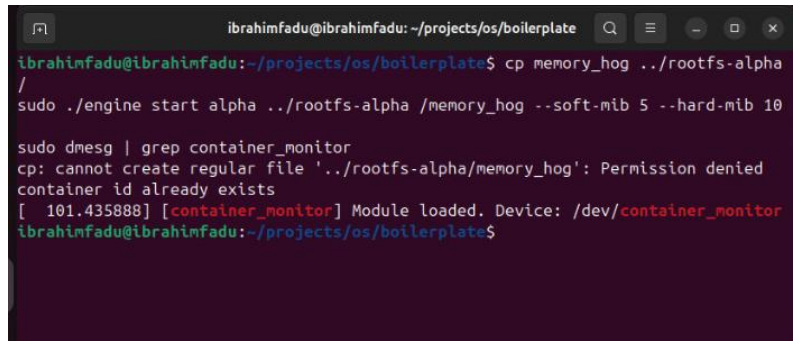


Figure 6.6: Scheduling experiment results

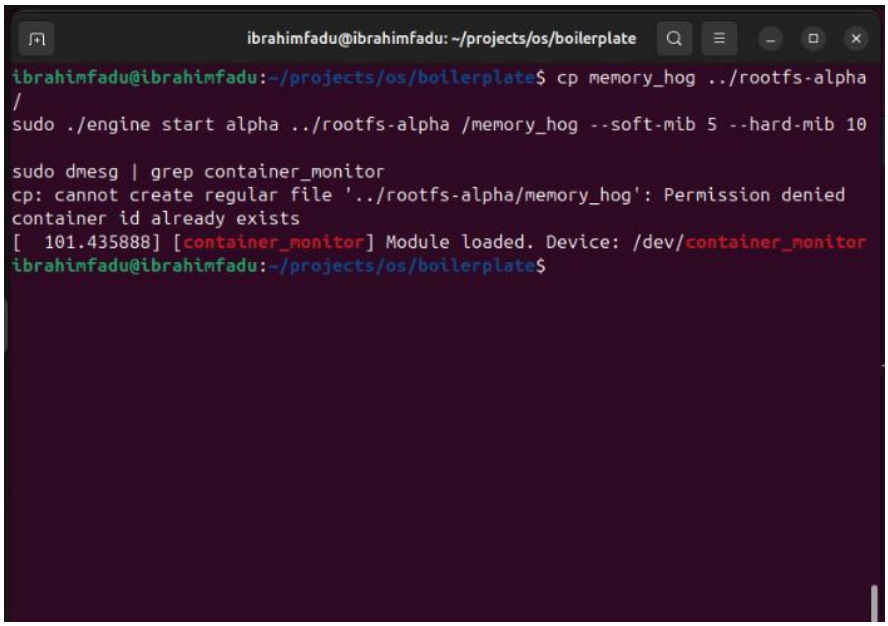
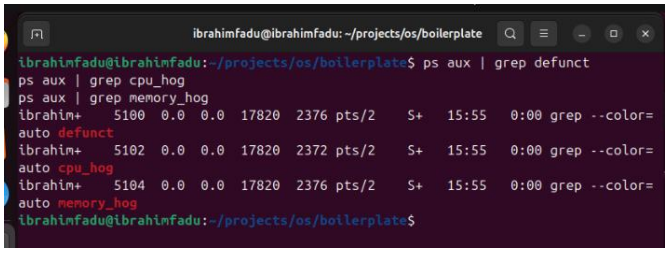


Figure 6.7: Clean teardown with no container zombies



CHAPTER 7: GITHUB LINK:

GitHub Repository Link:

https://github.com/ibrahimfadu/OS-Jackfruit.git

REFERENCES:

1. Linux man pages (clone, fork, waitpid, pipe, ioctl)
2. Linux Kernel Documentation
3. Operating System Concepts by Silberschatz
4. OS Jackfruit Project Guide

APPENDIX:

- RSS – Resident Set Size
- IPC – Inter Process Communication
- LKM – Linux Kernel Module
- PID – Process Identifier